

Асинхронный модуль - самообучение на LMS

ШАБЛОН РАЗРАБОТКИ СЦЕНАРИЯ ЗАНЯТИЯ

Вводные курса	
Название дисциплины	FastAPI Production: от кода до деплоя
Аудитория (для каких студентов)	Студенты 4 курса, выпускники, Junior Python-разработчики, прошедшие базовый курс по FastAPI
Длительность курса (ак.ч.)	4-6 недель (40-60 ак.ч.)
Среда (онлайн/оффлайн/гибрид)	Гибридная: LMS + GitHub + синхронные вебинары (раз в неделю)
Параметры аудитории	
Количество студентов	15-25 человек (ограничено для качественного код-ревью)
Уровень подготовки студентов (знание предмета)	Начально-средний: знание основ FastAPI, Python, Git, базовое понимание БД
Уровень мотивации (желание заниматься)	Высокий: трудоустройство, создание портфолио, переход на следующий уровень

Уровень технической подготовки (владение цифровыми инструментами и сервисами)	Уверенная работа с IDE, терминалом, Git, Docker (базово)
Дополнительные примечания, особенности аудитории	Опыт прохождения базового курса Потребность в обратной связи Ориентация на реальные проекты Готовность к групповой работе
Аутентичная задача обучения	
Разработать production-ready API с тестами, CI/CD пайплайном и деплоем на сервер, готовое для портфолио и демонстрации работодателю.	
Образовательная цель курса	
Подготовка к реальной разработке: production-ready код, тестирование, деплой, работа в команде, code review.	

Тема занятия	Тестирование FastAPI приложений с Pytest		
Образовательная цель занятия	Студент понимает стратегии тестирования веб-приложений и способен написать unit и	Формат проверки (образовательная цель достигнута, если)	Код студента проходит CI/CD пайплайн с покрытием тестов >80%, все

	integration тесты для своего API.		тесты зелёные.
Учебная задача 1	Изучить теоретичес кий блок «Виды тестов: unit, integration, e2e».	Формат проверки (учебная задача выполнена, если)	Тест из 10 вопросов (порог 90% правильн ых).
Учебная задача 2	Настроить pytest в проекте, написать 5 unit-тестов для сервисного слоя.	Формат проверки (учебная задача выполнена, если)	Pull Request в GitHub с passing tests.
Учебная задача 3	Написать 3 integration теста для API эндпоинто в с использова нием TestClient.	Формат проверки (учебная задача выполнена, если)	Код- ревью от ментора (одобрено без критическ их замечани й).

Общая структура занятия

	До занятия	В начале занятия	Основная часть	В конце занятия	После занятия
	подготовка	знакомство, погружение в контекст, настройка	практика и теория в разных форматах	рефлексия, (само)оценивание	практика/агрегация знаний
Учебная задача	Доступ к LMS	Видео-лекция (15 мин)	Написание тестов	Self-check чек-лист	Push в GitHub
Контент - теория, основное содержание, доп. материалы	Инструкция	Теория + примеры	Задачи на LMS	Критерии оценки	PR template
Механики - активности/тип заданий	Чек-лист	Видео + конспект	Код + автопроверка	Самооценка	Code review
Тайминг	До начала	20 мин	90 мин	30 мин	40 мин
Инструменты для реализации механик	Email, LMS	YouTube, Zoom	IDE, pytest	Google Form	GitHub, CI/CD

Сценарий занятия

Шаг	Что происходит	Время	Что нужно подготовить
До занятия			
1	Доступ открыт, письмо с инструкцией	-	Аккаунт на LMS, GitHub
2	Проверка окружения (pytest установлен)	15 мин	requirements.txt, venv
В начале занятия			
1	Просмотр видео-лекции «Стратегии тестирования»	20 мин	Видео на LMS, конспект
2	Изучение примеров тестов из репозитория курса	15 мин	GitHub repo course-examples
3	Заполнение pre-test опроса (уровень понимания)	10 мин	Google Form
Основная часть			
1	Настройка pytest.conf, фикстур, базовой структуры	30 мин	Шаблон проекта
2	Написание 5 unit-тестов для сервисов	40 мин	IDE, документация pytest
3	Написание 3 integration тестов для API	40 мин	TestClient, база данных для тестов
4	Запуск локально, отладка failing tests	30 мин	Терминал, логи

5	Commit + Push в feature branch, создание PR	20 мин	GitHub, template PR
В конце занятия			
1	Self-check по чек-листу (покрытие, naming, fixtures)	15 мин	Чек-лист в LMS
2	Отправка PR на ревью ментору	10 мин	GitHub PR
3	Запись вопросов для office hours (если есть)	10 мин	Discord, Issues
После занятия			
1	Исправление замечаний после код-ревью	2-3 дня	GitHub Comments
2	Переход к следующему модулю (CI/CD)	-	Ссылка на Module 3

Синхронный воркшоп - Live Code Review с ментором

ШАБЛОН РАЗРАБОТКИ СЦЕНАРИЯ ЗАНЯТИЯ

Вводные курса	
Название дисциплины	FastAPI Production: от кода до деплоя

Аудитория (для каких студентов)	Студенты, прошедшие Модуль 1-2 асинхронной части курса
Длительность курса (ак.ч.)	2 ак.ч. (онлайн-встреча)
Среда (онлайн/оффлайн/гибрид)	Онлайн, синхронно (Zoom + Discord + GitHub)
Параметры аудитории	
Количество студентов	До 15 человек (для индивидуального внимания)
Уровень подготовки студентов (знание предмета)	Начально-средний: написали тесты, есть PR на ревью
Уровень мотивации (желание заниматься)	Высокий: получение фидбека, исправление ошибок перед деплоем
Уровень технической подготовки (владение цифровыми инструментами и сервисами)	Уверенный: Git, Docker, IDE, GitHub Actions
Дополнительные примечания, особенности аудитории	Потребность в живой обратной связи Страх публичной демонстрации кода Разный темп выполнения ДЗ
Аутентичная задача обучения	
Получить экспертное код-ревью, устранить критические ошибки в тестах и архитектуре перед финальным деплоем.	

Образовательная цель курса

Подготовка к реальной разработке в команде: умение принимать фидбек, защищать архитектурные решения, работать по Git-flow.

Тема занятия	Code Review сессия - Тесты и Архитектура		
Образовательная цель занятия	Студент получает персонализированный фидбек по коду, понимает свои типовые ошибки и знает план исправлений.	Формат проверки (образовательная цель достигнута, если)	Студент зафиксировал замечания в Issue Tracker и внес правки в течение 48 часов после воркшопа.
Учебная задача 1	Подготовить ссылку на GitHub PR с тестами до начала воркшопа.	Формат проверки (учебная задача выполнена, если)	Ссылка в чате Discord за 24 часа до встречи.
Учебная задача 2	Презентовать своё решение (5 минут на студента, 3 студента в фокусе).	Формат проверки (учебная задача выполнена, если)	Демонстрация экрана, объяснение архитектурных решений.
Учебная задача 3	Получить и зафиксировать feedback от ментора и группы.	Формат проверки (учебная задача	Заполненный Issue с action items в

		выполнена, если)	репозитории.
--	--	------------------	--------------

Общая структура занятия					
	До занятия	В начале занятия	Основная часть	В конце занятия	После занятия
	подготовка	знакомство, погружение в контекст, сонастройка	практика и теория в разных форматах	рефлексия, (само)оценивание	практика/агрегация знаний
Учебная задача	Сбор PR ссылок	Приветствие, нетворкинг	Разбор кода студентов	Q&A сессия	План исправлений
Контент - теория, основное содержание, доп. материалы	GitHub PR	Zoom, микрофон	Shared Screen, IDE	Чат, опросы	Issue Tracker
Механики - активности /тип заданий	Google Form	Icebreaker	Live coding review	Открытый микрофон	Action items

Тайминг	До начала	15 мин	70 мин	25 мин	10 мин
Инструменты для реализации и механик	Discord, Email	Zoom	GitHub, IDE	Miro, Chat	GitHub Issues

Сценарий занятия			
Шаг	Что происходит	Время	Что нужно подготовить
До занятия			
1	Рассылка напоминания + ссылка на Zoom	1 день	Calendar invite, Zoom link
2	Сбор ссылок на GitHub PR от студентов	24 часа	Google Form, Discord
В начале занятия			
1	Приветствие, проверка связи, знакомство	10 мин	Zoom-комната, микрофон
2	Обозначение целей: разбор 3 проектов в фокусе	5 мин	Презентация с планом
3	Icebreaker: “Самая сложная ошибка в тестах”	10 мин	Чат, опрос Mentimeter
Основная часть			

1	Разбор типовых ошибок (на анонимных примерах)	20 мин	Подготовленные сниппеты кода
2	Code Review А Студент (архитектура тестов)	15 мин	Доступ к репозиторию, экран
3	Code Review Б (fixtures, mocking) Студент Б	15 мин	GitHub PR, комментарии
4	Code Review В (integration tests, DB) Студент В	15 мин	Test database, Docker
5	Групповое обсуждение: лучшие практики	15 мин	Miro board, чат
В конце занятия			
1	Подведение итогов: общие паттерны ошибок	10 мин	Слайд с summary
2	Индивидуальные action items для каждого	10 мин	GitHub Issues template
3	Сбор обратной связи по воркшопу	5 мин	Google Form (NPS)
После занятия			
1	Отправка записи + чек-лист исправлений	1 день после	YouTube, Email

2	Проверка исправленного кода (дедлайн 48 часов)	2 дня после	GitHub PR review
---	--	-------------	------------------